On the Mitigation of Controllable Event Triggering Attack in WSNs

Haoran Hu*† and Wei Chang*
*Department of Computer Science, Saint Joseph's University, Philadelphia, PA, USA 19131
†College of Information Science and Engineering, Hunan Normal University, Changsha, China 410081
Email: hhrh1h2r3@gmail.com, wchang@sju.edu

***Abstract*—Compressive sensing-based data collecting is a promising technique, which can significantly reduce the communication costs in wireless sensor networks (WSNs). However, a recent study shows that it is vulnerable to controllable event triggering attack (CETA): after compromising a sensor and carefully manipulating the environmental elements around a target in the subtree rooted at the sensor, an attacker is able to infer sensitive parameters of the target. The existing countermeasure is purely based on cryptography and requires central control. In this paper, we propose a lightweight non-cryptography-based approach by termly modifying the structure of the data gathering tree. However, how to efficiently and effectively construct the tree is an open problem. To solve the problem, we create a novel topology-based coding scheme and a distributed algorithm to mitigate the CETA attack. By adopting this approach, the time for successfully launching a CETA attack is significantly increased. Extensive simulations show that our solution can efficiently and securely build different data gathering trees for consecutive sensing tasks.**

***Index Terms*—Compressive Sensing, Event Triggering Attack, Topology Representation, Wireless Sensor Network**

I. Introduction

As smart cities are being widely built in many urban areas [1] [2], which use sensors to collect data to optimize infrastructure management, the security of smart cities has been brought into concern [3]. Data gathering, where data collection is accomplished through a tree-based multi-hop routing from sensors towards a common sink node, is a fundamental operation in WSNs. Directly routing individual reading from each sensor to the sink is the most straightforward approach, but it results in unbalanced energy consumption: the sensors closer to the sink will soon run out of energy.

Compressive data gathering (CDG) [4] is a data-gathering approach based on the compressive sampling theory, which reduces the communication costs by exploring the correlations among the sensing data so that the network lifetime is largely prolonged. Instead of sending individual data, each node sends out m weighted sums of the readings of all sensor nodes in its subtree. Eventually, the sink node obtains the weighted sums of all sensor readings, and then accurately reconstructs the original data.

However, CDG is subject to controllable event triggering attack (CETA) [5]. Before using CDG, every sensor node needs to preinstall a random coefficient vector ϕ_j, the values of which are sensitive. Suppose v_j is the target node, which is a member of the subtree rooted at a compromised sensor node v_i. By triggering a controllable event around v_j, such as increasing the surrounding temperature by Δx_i degree, the weighted sum returning through v_i increases by Δy_i accordingly. In this way, the attacker can infer ϕ_j by $\phi_j \approx \Delta y_i / \Delta x_j$.

Paper [5] provides a hash key chain-based countermeasure. In this solution, the sink acts as a central controller, and all sensors must update their coefficient vectors simultaneously. This means the key-based solution requires central control, and inevitably there are key management issues.

In this paper, we propose an approach to defer CETA by dynamically modifying the structure of the data gathering tree. The main idea is that the attacker cannot infer ϕ_j if he is unable to correctly obtain the value of Δy_i corresponding to a controllable event Δx_i. To achieve it, our approach changes the structure of every subtree in the whole data gathering tree in consecutive sensing rounds, such that the variance of sensor readings caused by a controllable event is hidden by the modifications of data gathering trees' memberships.

Due to the lack of global information in WSNs, how to securely coordinate the modifications of return paths among different sensors is an open problem. We face three challenges and solve them in this study. (1) How to present and store the topologic position information efficiently. (2) How to avoid forming a loop. (3) How to avoid redundant modifications to the data gathering tree under the premise of ensuring that the members of the subtree rooted at every node are different from that in the last round. To solve these challenges, we propose a novel form of topologic information representation and a new algorithm called Subtree Modification Algorithm (SMA), which is self-organized, distributed and non-cryptography-based.

The remainder of the paper is organized as follows: Section II introduces the background knowledge about CDG in WSNs and CETA. Section III illustrates our system model on which our solution is based. Section IV details the implementation of SMA and the solutions to the three aforementioned challenges. Section V provides comprehensive evaluation results. Section VI concludes the paper.

978-1-7281-6607-0/20/$31.00 ©2020 IEEE
Authorized licensed use limited to: Auckland University of Technology. Downloaded on November 14,2020 at 10:26:20 UTC from IEEE Xplore. Restrictions apply.

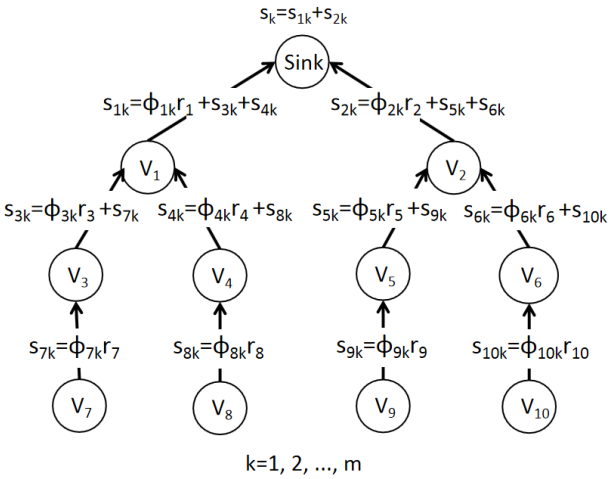


Figure 1: Compressive Data Gathering

II. Preliminary Work

In compressive data gathering, each sensing round can be divided into m forwarding rounds because it takes each sensor m rounds to forward a single reading to the sink using different coefficients. Denote the coefficient vector preinstalled in v_i as $(\phi_{i1}, \phi_{i2}, ..., \phi_{im})$. Fig. 1 shows a simple compressive sensing-based data gathering tree at the kth sensing round. Denote the reading of v_i as r_i. Denote the weighted sum calculated by v_i in the kth forwarding round as s_{ik}. Denote the coefficient used by v_i in the kth forwarding round as ϕ_{ik}. Take the route $v_7 \rightarrow v_3 \rightarrow v_1 \rightarrow Sink$ as an example. v_7 is a leaf node. v_7 multiplies its reading r_7 by ϕ_{7k} and then gains s_7. v_7 forwards s_7 to v_3. v_3 multiplies r_3 by ϕ_{3k} and pluses the received s_7 to gain s_3. v_3 forwards s_3 to v_1. v_1 sums up s_3 and s_4 received from v_3 and v_4 respectively and pluses its weighted reading $\phi_{1k}r_1$ to gain s_1. Eventually, the sink adds s_{1k} and s_{2k} up and then gains the weighted sum of all sensor readings for the kth forwarding round.

After all sensors repeated the process m times using different sets of coefficients, the sink receives m weighted sums of all sensor readings for the current sensing round. The sink has a copy of all coefficients of all sensors. The goal of the sink is to recover n readings from m weighted sums, $\sum_{i=1}^{n} \phi_{ik} r_i$ $(k = 1, 2, ..., m)$, where $m<n$. Since the sensor readings are correlated, based on the pre-learned correlation information, the sink can recover the original reading of every sensor.

However, CDG is vulnerable to CETA. Note that in different sensing rounds but the same forwarding round, a node uses the same coefficient. In Fig. 1, suppose v_1 is compromised, the attacker can obtain the weighted sums of r_3, r_4, r_7 and r_8, which are propagated through v_1. Suppose the attacker takes v_7 as the target. He triggers a controllable event, like a fire if v_7 is a temperature sensor, around v_7. Then r_7 increases by Δr_7. The s_{1k} increases by Δs_{1k}. Suppose there is no significant natural event happening around v_3, v_4 and v_8. So Δs_{1k} is mainly caused by Δr_7, that is, $\Delta s_{1k} \approx \phi_{7k}\Delta r_7$. The attacker infers ϕ_{7k} by $\phi_{7k} \approx \Delta s_{1k}/\Delta r_7$ $(k = 1, 2, ..., m)$. Later, by launching CETA targeting at v_3, v_4 and v_8 one by one, the attacker infers the coefficient vectors of all sensors in the subtree rooted at v_1.

Even though sometimes sensor reading r_i can vary significantly without any controllable event being triggered by the attacker, the attacker can reduce its impact on the precision of his calculated coefficient by increasing the number of observations and using some statistical methods.

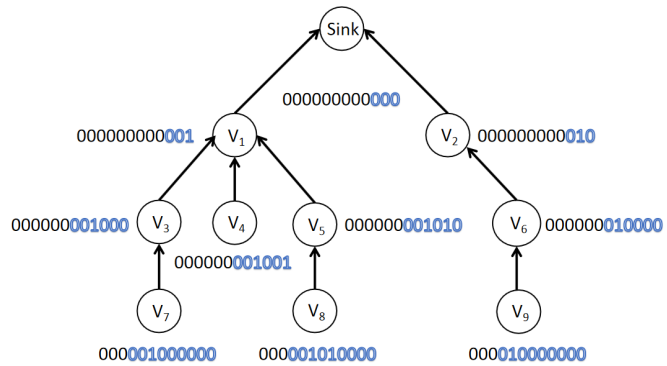


Figure 2: System Model

III. System Model

Fig. 2 shows a data gathering tree in a WSN. v_i and v_j are neighbors only if v_i and v_j can communicate bidirectionally. Denote the set of all nodes in the subtree rooted at v_i as $S(v_i)$. For example, in Fig.2, $S(v_1) = \{v_3, v_4, v_5, v_7, v_8\}$. Denote the parent node of v_i as $P(v_i)$.

In our scheme, SMA, $S(v_i)$ of every node v_i changes every sensing round. Suppose an attacker has compromised v_1. In the jth forwarding round of the ith sensing round, the weighted sum gained by the attacker is $\phi_{3j}r_3 + \phi_{4j}r_4 + \phi_{5j}r_5 + \phi_{7j}r_7 + \phi_{8j}r_8$. However, in the $(i+1)^{th}$ sensing round, $S(v_1)$ changes by running SMA. Suppose $S(v_1)$ changes to $\{v_3, v_4, v_5, v_7\}$. In the jth forwarding round of the $(i+1)^{th}$ sensing round, the weighted sum gained by the attacker is $\phi_{3j}r_3 + \phi_{4j}r_4 + \phi_{5j}r_5 + \phi_{7j}r_7$.

In this paper, we design a topology-based coding scheme. Every node has a pair of Fixed ID and Topology ID. The Fixed ID is fixed. The Topology ID is presented in the form of a binary string. Denote the Topology ID of v_i as $TID(v_i)$. $TID(v_i)$ contains the topologic position information of v_i in bits at the right side and has its left side padded with 0. The length of the right side of $TID(v_i)$ is equal to the product of the hops from v_i to the sink timing the value of a parameter, k.

The length of TID is a parameter, l. The value of l is assigned according to the maximum number of the sensor nodes that a network can accommodate. Suppose there can be at most n nodes in the network. To make full use of TID,

Authorized licensed use limited to: Auckland University of Technology. Downloaded on November 14,2020 at 10:26:20 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Subtree Modification Algorithm (SMA)

1: **for** $i = 0$ to n **do**
2: $F_{fin}(v_i) \leftarrow false$
3: **if** v_i is not a leaf **then**
4: $F_{nc}(v_i) \leftarrow true$
5: **else**
6: $F_{nc}(v_i) \leftarrow false$
7: **end if**
8: **end for**
9: **for** $i = 0$ to n **do**
10: **if** $F_{nc}(v_i) = true$ && $F_{fin}(v_i) = false$ **then**
11: $depth \leftarrow 1$
12: Ask $L(v_i, depth)$ if there is any node that can move out from $S(v_i)$
13: **while** Receive no response **do**
14: $depth \leftarrow depth + 1$
15: Ask $L(v_i, depth)$ if there is any node that can move out from $S(v_i)$
16: **end while**
17: Choose a responder to move out from $S(v_i)$
18: Update F_{fin}, TID of all relative nodes
19: **end if**
20: **end for**

we have $2^{l-1} < n \leq 2^l \implies \log_2 n \leq l < \log_2 n + 1$. So we assign the integer $\in [\log_2 n, \log_2 n + 1)$ to l. Our coding scheme requires that: (1) Every non-sink node can have at most 2^k child nodes. (2) The sink can have at most $2^k - 1$ child nodes, of which the reason will be addressed in the second paragraph of Section IV Subsection B. (3) The maximum depth of the data gathering tree is l/k. The value of k is assigned according to the maximum number of the child nodes that every non-sink node can have and the maximum depth of the data gathering tree. For all examples in the remaining paper, $l = 12$, $k = 3$, so there can be at most 4048 sensor nodes in the network, the depth of the data gathering tree can be at most 4, and each node can have at most 8 child nodes. For simplicity and convenience, in the remaining part of this paper, when it comes to TID, we refer to the TID without the sequence of padded leading zeros.

IV. Data Gathering Tree Reconstruction

A. Subtree Modification Algorithm

During each sensing round, every sensor runs SMA before the sensor readings are gathered. Alg. 1 illustrates the general process of SMA.

Alg. 1 lines 1-8 initialize the procedure. Every non-sink node v_i has a $F_{nc}(v_i)$ flag and a $F_{fin}(v_i)$ flag. At the beginning of the sensing round, every node v_i sets its $F_{fin}(v_i)$ to false and then checks if it is a non-leaf node. If it is, which means $S(v_i)$ is not empty now so that $S(v_i)$ needs to be changed during the adoption of SMA, v_i sets its $F_{nc}(v_i)$ to true. $F_{nc}(v_i)$ is set to false otherwise. $F_{fin}(v_i)$ is set to true if $S(v_i)$ is changed as SMA runs.

Algorithm 2 Generation of $TID(C(v_i))$

if Receive $TID(v_i)$ from $P(v_i)$ **then**
for $j = 0$ to $C(v_i).length - 1$ **do**
if v_i is the sink && $j = 0$ **then**
$j \leftarrow j + 1$
end if
$TID(C(v_i, j)) \leftarrow TID(v_i) << k + j$
Send $TID(C(v_i, j))$ to $C(v_i, j)$
end for
end if

Alg. 1 lines 9-10 determine whether to change the tree structure. Denote the set of all nodes at the jth layer in the subtree rooted at v_i as $L(v_i, j)$. For example, in Fig. 2, $L(v_1, 1)$ is $\{v_3, v_4, v_5\}$. Suppose now v_1 is determining whether it needs to change $S(v_1)$. v_1 finds that $F_{nc}(v_1) = true$ && $F_{fin}(v_1) = false$. $F_{nc}(v_1) = true$ means $S(v_1)$ needs to be changed while SMA is running. $F_{fin}(v_1) = false$ means since SMA in the current round began, none of the previous modifications done by other nodes changes $S(v_1)$. So v_1 needs to change $S(v_1)$ by itself now.

Alg. 1 lines 11-17 modify the subtree structure. v_1 first asks $L(v_1, 1)$ if there is any node v_j that can move out from $S(v_1)$ by changing $P(v_j)$ to a neighboring node. The neighboring node needs to meet the following three conditions: (1) it is outside $S(v_1)$. (2) Its depth in the data gathering tree allows it to have a child node since the maximum depth of the data gathering tree is l/k. (3) It has less than 8 child nodes, that is, it can accommodate at least one new child node. Suppose there is no node in $L(v_1, 1)$ that can move out from $S(v_1)$, so v_1 receives no response. Then v_1 asks the remaining layers one by one in the depth-increasing order until it receives at least one response from the last queried layer. Suppose both v_7 and v_8 in $L(v_1, 2)$ respond an offer message to v_1. Every node v_i can memorize all $S(v_i)$ in the last m sensing rounds and accepts the offer message sent by the node that can bring it an $S(v_i)$ that is new compared with all subtrees in last m sensing rounds, where m is a parameter. If no node can, v_i will randomly accept one. In this example, v_1 predicts that $S(v_1)$ will become $\{v_3, v_4, v_5, v_8\}$ if it chooses v_7 to move out from $S(v_1)$, and $S(v_1)$ will become $\{v_3, v_4, v_5, v_7\}$ if it chooses v_8 to move out from $S(v_1)$. Suppose v_1 finds that $S(v_1)$ was $\{v_3, v_4, v_5, v_8\}$ in one round of last m sensing rounds while $\{v_3, v_4, v_5, v_7\}$ is new by looking up its memory. So v_1 accepts v_8's offer message and sends an acknowledge message to v_8. v_8 changes $P(v_8)$ from v_5 to v_9. So $S(v_1)$ is changed from $\{v_3, v_4, v_5, v_7, v_8\}$ to $\{v_3, v_4, v_5, v_7\}$.

Alg. 1 lines 18-20 update flags. Finally, F_{fin}, TID of all relative nodes get updated.

B. Generation of Topology ID

When a data gathering tree just initialized, TIDs of all nodes need to be initialized. When TID of a node changed,

Authorized licensed use limited to: Auckland University of Technology. Downloaded on November 14,2020 at 10:26:20 UTC from IEEE Xplore. Restrictions apply.

TIDs of all its descendant nodes need to be updated. Alg. 2 illustrates the generation of $TID(C(v_i))$, where we denote the jth child node of v_i as $C(v_i, j)$. TID of every non-sink ancestor node of v_i is a prefix of $TID(v_i)$. $TID(Sink)$ is 0, which stays the same all the time. Take the generation of the TID of v_3, v_4 and v_5 in Fig. 2 as an example. v_1 left shifts $TID(v_1)$ 3 bits, and hence 000000001000. Then v_1 adds 000, 001 and 010 to 000000001000 respectively, and hence $TID(v_3)$, $TID(v_4)$ and $TID(v_5)$, which are 000000001000, 000000001001, and 000000001010 respectively. v_1 sends the calculated $TID(v_3)$, $TID(v_4)$ and $TID(v_5)$ to v_3, v_4 and v_5 respectively. Then v_3 and v_5 begin to calculate $TID(v_7)$ and $TID(v_8)$ respectively.

The following example illustrates the reason why the sink can have at most 2^k-1 child nodes instead of 2^k child nodes. In Fig. 2, suppose $TID(v_1)$ is not 000000000001 but 000000000000. Then through the aforementioned calculation, $TID(v_3)$, $TID(v_4)$ and $TID(v_5)$ are 000000000000, 000000000001 and 000000000010 respectively. $TID(v_3)$ is the same as $TID(v_1)$ and $TID(sink)$. $TID(v_5)$ is the same as $TID(v_2)$. As a result, TID loses uniqueness. We can conclude that none of the child nodes of the sink can have 000000000000 as its TID, which means the sink can only have at most 2^k-1 child nodes.

C. Loop-free Scheme

Denote the old parent node and the new parent node of v_i as $P_{old}(v_i)$ and $P_{new}(v_i)$ respectively. Suppose v_j is the ancestor node of v_i that lets v_i change $S(v_j)$. To achieve the loop-free scheme, the following two conditions need to be met: (1) $P_{new}(v_i) \notin S(v_i)$. (2) When v_i is changing $P(v_i)$, there cannot be any other nodes changing their parent nodes.

As we have mentioned in the fourth paragraph of Section IV, when v_i changes $P(v_i)$ from $P_{old}(v_i)$ to $P_{new}(v_i)$ to help v_j to change $S(v_j)$, v_i has already made sure that $P_{new}(v_i) \notin S(v_j)$. Also, since v_j is the ancestor node of v_i, we have $S(v_i) \subset S(v_j)$. Based on the two conditions, we get $P_{new}(v_i) \notin S(v_i)$. So the first condition is met in SMA. However, the second condition is not met yet.

We illustrate why the second condition is necessary for the loop-free scheme by showing how a loop forms when multiple nodes update their parent nodes simultaneously in Fig. 3. Denote all neighboring nodes of v_i as $N(v_i)$. Suppose every node v_i randomly chooses a moment to change $S(v_i)$. Before the modifications to the data gathering tree take place, v_4, v_5 and v_6 are in the same layer. At the same moment, v_1 decides to change $S(v_1)$, v_2 decides to change $S(v_2)$, and v_3 decides to change $S(v_3)$. v_1 selects v_4 to move out from $S(v_1)$, v_2 selects v_5 to move out from $S(v_2)$, and v_3 selects v_6 to move out from $S(v_3)$. v_4 chooses v_5 as $P_{new}(v_4)$ after making sure that $v_5 \in N(v_4)$ and $v_5 \notin S(v_1)$ and $depth(v_5) < 4$ and $C(v_5).length < 8$. Similarly, v_5 chooses v_6 as $P_{new}(v_5)$, and v_6 chooses v_4 as $P_{new}(v_4)$. In this case, a loop forms without v_4, v_5 or v_6's notice. As a result, v_4, v_5 and v_6 are cut off from the data gathering tree and keep forwarding readings among themselves.

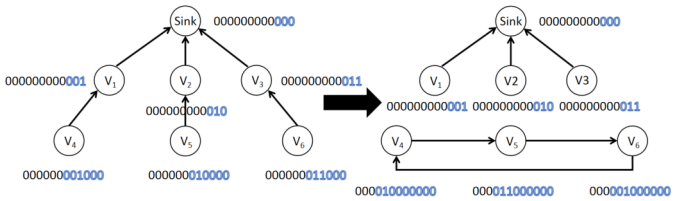


Figure 3: Loop

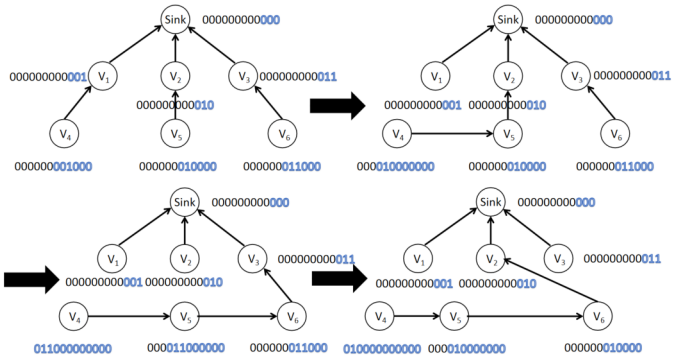


Figure 4: Loop-free

Fig. 4 shows how the loop-free scheme is achieved when the second condition is also met. Note that once v_i changed $P(v_i)$ and updated $TID(v_i)$, all nodes in $S(v_i)$ will be informed of updating their TIDs. After v_4 chose v_5 as $P_{new}(v_4)$ and v_5 chose v_6 as $P_{new}(v_5)$, TIDs of v_4 and v_5 are updated. When v_6 is choosing a node to be $P_{new}(v_6)$, it finds that $TID(v_3)$ is a prefix of $TID(v_4)$, which indicates that $v_4 \in S(v_3)$. So v_6 chooses v_2 instead of v_4 as $P_{new}(v_6)$.

To meet the second condition, we have to specify an order so that every node v_i changes $P(v_i)$ one by one by following this order. We consider two orders. The first order is a pre-fixed order which adopts the time slot. We divide the time which is used to adopt SMA during each sensing round into many time slots. We assign each node a dedicated time slot. During the dedicated time slot of v_i, v_i can decide whether $P(v_i)$ needs to be changed or not, depending on if $F_{nc}(v_i) = true$ && $F_{fin}(v_i) = false$, and v_i can change $P(v_i)$ if $P(v_i)$ needs to be changed by v_i. Other nodes cannot change their parent nodes at this time slot. The second order is a topology-based order which adopts pre-order traversal. v_i is informed by $P(v_i)$ of changing $S(v_i)$ and informing all of its child nodes of changing theirs. v_i won't reply to $P(v_i)$ that it has finished until it changes $S(v_i)$, informs all of its child nodes of changing theirs and receives their replies. v_i won't inform the next child node until it receives the reply of the last informed child node.

D. Avoiding Redundant Modifications

In Fig. 5, suppose v_3 informs v_7 of changing $P(v_7)$ to a node that is outside $S(v_3)$. When v_7 changes $P(v_7)$ from v_5 to v_6, it changes not only $S(v_3)$ but also $S(v_4)$, $S(v_5)$ and $S(v_6)$. So for every node v_i among v_4, v_5 and v_6, if the turn

Authorized licensed use limited to: Auckland University of Technology. Downloaded on November 14,2020 at 10:26:20 UTC from IEEE Xplore. Restrictions apply.

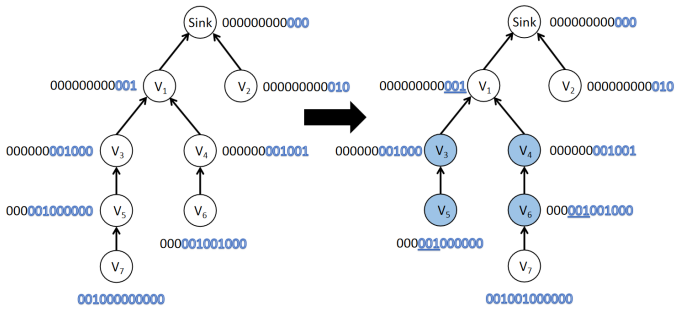
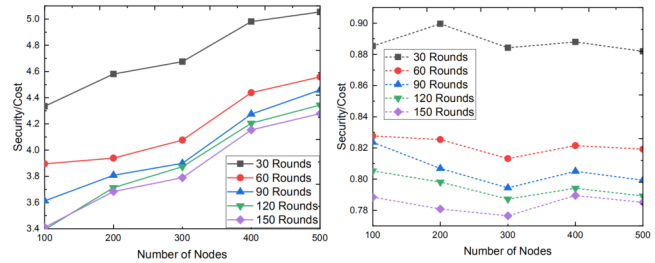


Figure 5: Update F_{fin}



(a) SMA (b) LPCA

Figure 6: Number of Nodes

for it to change $S(v_i)$ has not come yet, it doesn't need to change $S(v_i)$ anymore. However, $S(v_1)$ remains unchanged.

Denote all ancestor nodes of v_i as $A(v_i)$. Denote the old parent node and the new parent node of v_7 as $P_{old}(v_7)$ and $P_{new}(v_7)$ respectively. We can see that $\{v_3, v_4, v_5, v_6\}$ is $A(P_{old}(v_7)) \cup A(P_{new}(v_7)) - A(P_{old}(v_7)) \cap A(P_{new}(v_7))$, and $\{v_1\}$ is $A(P_{old}(v_7)) \cap A(P_{new}(v_7))$. So we can conclude that as v_i changes $P(v_i)$, $S(v_a)$ changes, where $v_a \in A(P_{old}(v_i)) \cup A(P_{new}(v_i)) - A(P_{old}(v_i)) \cap A(P_{new}(v_i))$, and $S(v_b)$ remains unchanged, where $v_b \in A(P_{old}(v_i)) \cap A(P_{new}(v_i))$. The node located at the border of $A(P_{old}(v_i)) \cup A(P_{new}(v_i)) - A(P_{old}(v_i)) \cap A(P_{new}(v_i))$ and $A(P_{old}(v_i)) \cap A(P_{new}(v_i))$ is the crucial node, and we denote it as $Cru(v_i)$. $TID(Cru(v_i))$ is the longest common prefix of $TID(P_{old}(v_i))$ and $TID(P_{new}(v_i))$, of which the length is a multiple of k, where k is the aforementioned parameter whose value is assigned 3 in this paper. In Fig. 5, $Cru(v_7)$ is v_i.

In Fig. 5, v_7 sends an inform message carrying $TID(v_1)$ to v_5 and v_6. v_5 and v_6 forward the message to v_3 and v_4 respectively. v_3, v_4, v_5 and v_6 update their F_{fin} to true on the receipt of the inform message. When v_1 receives the message, it finds that its TID in the message, which indicates that the members of any subtree rooted at it or any of its ancestor nodes didn't change when v_7 changed $P(v_7)$. So v_1 discards the message. And the updating process of F_{fin} comes to the end.

Clearly, the $Cru(v_i)$ will receive two copies of the inform message while all other nodes receiving the inform message will only receive one copy. The reason why we use $TID(Cru(v_i))$ embedded in the inform message instead of the number of received inform messages to indicate that if a node is the $Cru(v_i)$ is that the $Cru(v_i)$ won't necessarily receive both two copies of the inform message at the same time. If it hasn't received the other copy of the inform message after waiting for a long time, it may assume that it is not the $Cru(v_i)$ and then forward the inform message to its parent node but receive the other copy later. This will cause the F_{fin} of nodes in $A(P_{old}(v_i)) \cap A(P_{new}(v_i))$ to be mistakenly updated to true.

V. Simulation and Analysis

As Section IV Subsection D indicates, the deeper the depth of the node v_i which changed $P(v_i)$ is, the larger $A(P_{old}(v_i)) \cup A(P_{new}(v_i)) - A(P_{old}(v_i)) \cap A(P_{new}(v_i))$ may be, the fewer modifications have to be done to the topology of the data gathering tree. So we create a relatively simpler algorithm called Leaf Parent Changing Algorithm (LPCA) to be compared with SMA. In LPCA, when it comes to v_i's time slot, v_i needs to change $P(v_i)$ if and only if it is a leaf node currently. If v_i needs to change $P(v_i)$, v_i will randomly choose a neighboring node whose depth in the data gathering tree allows it to have child nodes and which has less than 8 child nodes (that is, can accommodate a new child node) to be $P_{new}(v_i)$.

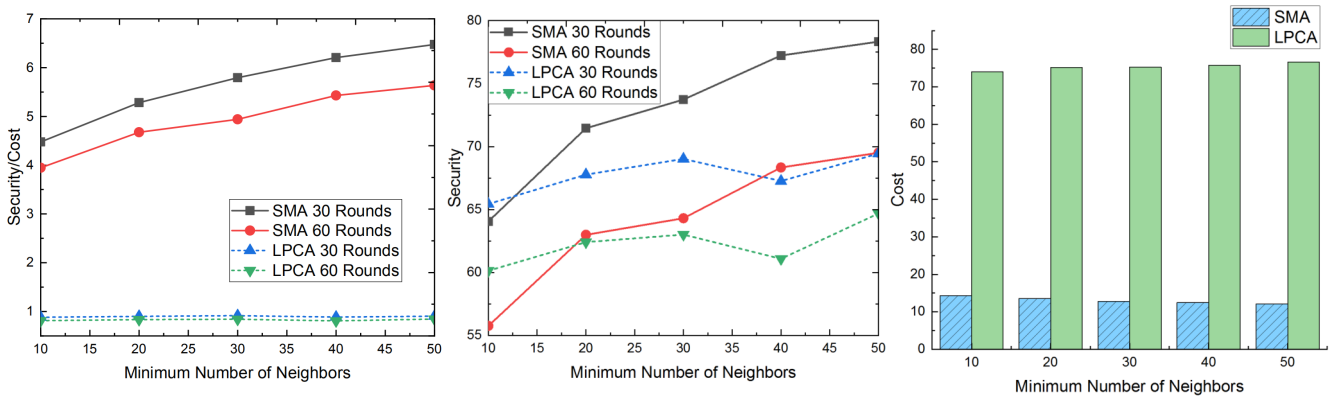
In our simulation, we run SMA and LPCA on two copies of the same randomly generated data gathering tree, where the signal range and the coordinate of every node are randomly generated, for 150 sensing rounds. The observation window is 30 sensing rounds. After each complete observation window, we collect the average data for both $security$ and $cost$ for the latest 30 sensing rounds. So we observe 5 times during 150 sensing rounds. For each chosen value of the parameter to be plotted as the x-axis on the graph, we run SMA and LPCA on 20 different randomly generated data gathering trees and calculate the average value of the results.

As the attacker only needs to monitor the weighted sum from a single subtree, we assume that he has enough memory to record the weighted sums for all sensing rounds. Denote the number of node v_i whose $F_{nc}(v_i) = true$ and whose $S(v_i)$ has been changed to an $S(v_i)$ which is new in all previous sensing rounds as s_1. Denote the number of node v_i whose $F_{nc}(v_i) = true$ as s_2. Denote the number of node v_i which changed $P(v_i)$ as c_1. Denote the total number of nodes as c_2. We evaluate the $security$ as s_1/s_2 (unit:%) and evaluate the $cost$ as c_1/c_2 (unit:%). We evaluate the performance of both SMA and LPCA as $security/cost$.

Standard parameters: number of nodes=300, the minimum number of neighbors=10, sensing area=100x100 (that is, can accommodate 10000 nodes), the capacity of memory for every node to record previous subtrees=10 rounds, the maximum number of child nodes=8, length of TID=30.

From the vertical direction of Fig. 6, we can see that as

Authorized licensed use limited to: Auckland University of Technology. Downloaded on November 14,2020 at 10:26:20 UTC from IEEE Xplore. Restrictions apply.



(a) Security/Cost (b) Security (c) Cost

Figure 7: Minimum Number of Neighbors

the duration increases, the performance of both SMA and LPCA goes down since the solution space of new subtrees is consumed and the possibility of repeating the subtree increases. As a result, the possibility for the attacker to infer the coefficients increases as time elapses.

From the horizontal direction of Fig. 6, we can see that as the number of nodes increases, the performance of SMA increases because the more nodes the data gathering tree has, the larger the density of nodes is, the more neighbors every node can have, the more nodes which reply an offer message to the ancestor node can be, the more options the ancestor node can have when choosing a node to move out from the subtree, the less the repetitive rate based on the subtrees in all previous sensing rounds can be, the higher the $security$ is.

The minimum number of neighbors is a parameter used to keep each node from being isolated from the whole data gathering tree. A node of which the number of neighbors is less than the allowed minimum value will take initiative to increase its range to be able to reach more nodes. From the vertical direction of Fig. 7 (a), we can see that SMA performs much better than LPCA. From the vertical direction of Fig. 7 (b), we can see that the $security$ of LPCA is almost as good as that of SMA. From the horizontal direction of Fig. 7 (a) (b), we can see that as the minimum number of neighbors goes up, the performance ($security/cost$) and the $security$ of SMA increases, of which the reason is similar to the aforementioned reason for the increment of performance as the number of nodes goes up. Fig. 7 (c) shows that the $cost$ of SMA is much lower than that of LPCA since SMA is stateful and follows a series of rules to avoid redundant modifications to the data gathering tree while LPCA is stateless and changes the parent node randomly.

VI. Conclusion

WSNs play an essential role in smart cities. CDG in WSNs can improve communication efficiency and network lifetime. However, CDG is vulnerable to CETA. In this paper, we find that having a fixed subtree membership of a compromised node is the prerequisite for launching a CETA attack; otherwise, the difference before and after a controllable event will not be able to be fully observed at the compromised node. Therefore, for a series of sensing tasks towards the sink node, the CETA attack is useless if the members of every subtree in the data gathering tree are always different. In this paper, we depict a novel form of topologic information representation and illustrate a self-organized, distributed and non-cryptography-based algorithm SMA, which continuously changes the members of every subtree in the data gathering tree. Extensive simulation results show that our approaches can efficiently and effectively increase the security in the compressive sensing-based data gathering process. In our future work, we will study how to further reduce the communication cost since there are multiple kinds of packets consuming energy and other resources in our approach.

References

[1] Leonidas Anthopoulos. Smart utopia vs smart reality: Learning by experience from 10 smart city cases. *Cities*, 63:128–148, 2017.

[2] Bhagya Nathali Silva, Murad Khan, and Kijun Han. Towards sustainable smart cities: A review of trends, architectures, components, and open challenges in smart cities. *Sustainable Cities and Society*, 38:697–713, 2018.

[3] Kuan Zhang, Jianbing Ni, Kan Yang, Xiaohui Liang, Ju Ren, and Xuemin Sherman Shen. Security and privacy in smart city applications: Challenges and solutions. *IEEE Communications Magazine*, 55(1):122–129, 2017.

[4] Chong Luo, Feng Wu, Jun Sun, and Chang Wen Chen. Compressive data gathering for large-scale wireless sensor networks. In *Proceedings of the 15th Annual International Conference on Mobile Computing and Networking*, MobiCom '09, pages 145–156, New York, NY, USA, 2009. Association for Computing Machinery.

[5] P. Hu, K. Xing, X. Cheng, H. Wei, and H. Zhu. Information leaks out: Attacks and countermeasures on compressive data gathering in wireless sensor networks. In *IEEE INFOCOM 2014 - IEEE Conference on Computer Communications*, pages 1258–1266, April 2014.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on November 14,2020 at 10:26:20 UTC from IEEE Xplore. Restrictions apply.